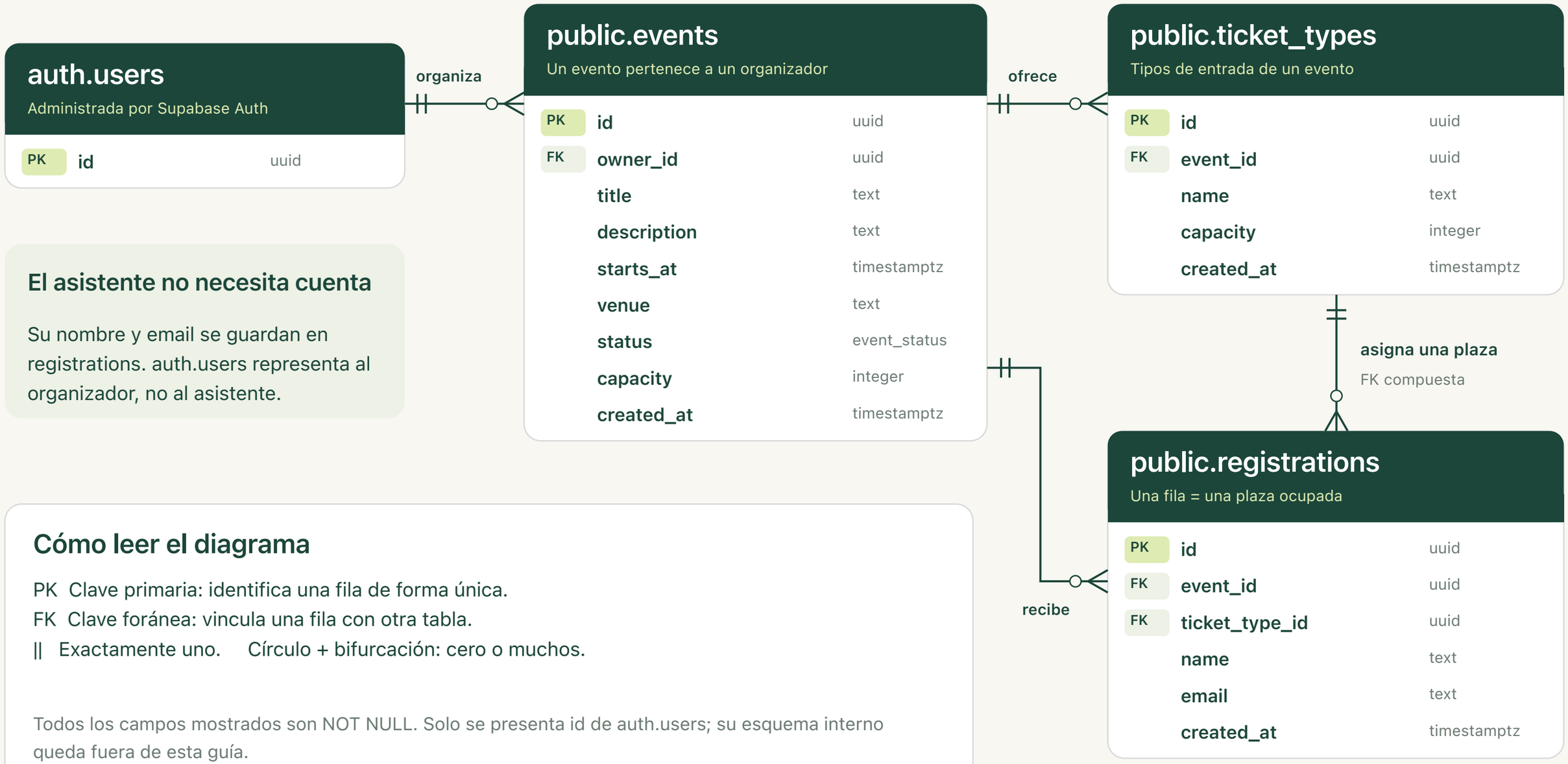
Mapa de entidades y relaciones

Un organizador crea eventos; cada evento ofrece entradas y recibe registros de asistentes.



El asistente no necesita cuenta

Su nombre y email se guardan en registrations. auth.users representa al organizador, no al asistente.

Cómo leer el diagrama

PK Clave primaria: identifica una fila de forma única.
FK Clave foránea: vincula una fila con otra tabla.
|| Exactamente uno. Círculo + bifurcación: cero o muchos.

Todos los campos mostrados son NOT NULL. Solo se presenta id de auth.users; su esquema interno queda fuera de esta guía.

Las relaciones conectan entidades; el detalle exacto de las FK está en la página 2.

Claves, restricciones y reglas de aforo

La integridad se protege con claves, CHECK, restricciones UNIQUE y triggers en PostgreSQL.

01 Relaciones y eliminación

events.owner_id → auth.users.id No permite eliminar un organizador con eventos.	RESTRICT
ticket_types.event_id → events.id Eliminar un evento elimina sus entradas, si no hay registros.	CASCADE
registrations.event_id → events.id Un evento con registros no se puede eliminar.	RESTRICT
registrations(event_id, ticket_type_id) → ticket_types(event_id, id) FK compuesta: la entrada debe pertenecer al evento; protege entradas con registros.	RESTRICT

02 Identidad y datos obligatorios

- PK: UUID generado automáticamente para cada fila pública.
- UNIQUE(event_id, email): un registro por email y evento.
- UNIQUE(event_id, id) en entradas: destino de la FK compuesta.
- Estado: enum draft | published; inicia en draft.
- Textos de evento y nombre de entrada: no vacíos ni solo espacios.
- Nombre del asistente: entre 1 y 200 caracteres tras trim.
- Email: minúsculas, trim, máximo 254 y formato básico válido.
- created_at: fecha de creación automática con now().

03 Aforo y disponibilidad

- Capacidad de evento y entrada: entero mayor que cero.
- La suma de capacidades de entradas no supera la del evento.
- El aforo del evento no baja de las plazas ya asignadas.
- Una entrada no baja de sus registros existentes.
- Una entrada con registros no puede cambiar de evento.

Cupos = capacidad del tipo – registros del tipo

Ejemplo: General 80 – 25 registros = 55 cupos disponibles.

Flujo de registro y acceso a los datos

La consulta informa disponibilidad; la transacción de registro toma la decisión final.

1 Preparar el evento

El organizador crea events, distribuye el aforo en ticket_types y publica el evento.

→

2 Consultar disponibilidad

get_public_registration_event devuelve datos públicos y cupos agregados. Sin nombres ni emails.

→

3 Completar el formulario

El asistente elige una entrada e ingresa nombre y email. Se valida en cliente y servidor.

4 register_attendee: una sola transacción

Bloquear la fila del evento → validar publicación y fecha → comprobar duplicado → verificar entrada y cupo → insertar

Si una validación falla: devuelve el motivo y no crea el registro. Si tiene éxito: una nueva fila ocupa una plaza.

El bloqueo serializa registros del mismo evento y evita que dos solicitudes consuman la última plaza.

Vigencia: fecha del evento ≥ hoy en America/Guayaquil. Incluye todo el día, aunque ya haya pasado la hora de inicio.

Permisos y privacidad (RLS)

- Público: lee eventos y entradas publicados.
- Organizador: administra solo sus eventos y entradas.
- Registros: sin lectura pública ni inserción directa para anon o authenticated.
- Propietario: puede leer registros de sus propios eventos.
- Las dos RPC públicas usan SECURITY DEFINER y search_path vacío; solo exponen la operación prevista.

Índices y notas de referencia

Índices: events(owner_id); events(starts_at) parcial para publicados; ticket_types(event_id); registrations(ticket_type_id). PK y UNIQUE también crean índices.

La política general de lectura de publicados no filtra fechas pasadas. Las RPC de registro sí aplican la vigencia. No existe una columna de cupos restantes.

Fuentes: supabase/migrations/20261104120000_events_and_ticket_types.sql · 20261104130000_event_capacity.sql · 20261106120000_public_registration.sql